設定Astra控制中心

Astra Control Center 22.08 documentation

NetApp
November 22, 2022

This PDF was generated from https://docs.netapp.com/zh-tw/astra-control-center-2208/get-started/add-cluster-reqs.html on November 22, 2022. Always check docs.netapp.com for the latest.

目錄

設定Astra控制中心 1
新增Astra Control Center授權 1
新增叢集 2
新增儲存後端 3
新增儲存庫 6
變更預設儲存類別 7
接下來呢？ 8
新增叢集的必要條件 8
新增自訂TLS憑證 13
建立自訂Pod安全性原則 17

設定Astra控制中心

Astra Control Center支援ONTAP 並監控將支援及Astra Data Store做為儲存後端。安裝Astra Control Center、登入UI並變更密碼之後、您將需要設定授權、新增叢集、管理儲存設備及新增儲存區。

工作

- a license for Astra Control Center
- cluster
- a storage backend
- a bucket

新增Astra Control Center授權

您可以使用UI或新增授權 "API" 以獲得完整的Astra控制中心功能。若無授權、您使用Astra Control Center的使用僅限於管理使用者及新增叢集。

如需如何計算授權的詳細資訊、請參閱 "授權"。

> 若要更新現有的評估或完整授權、請參閱 "更新現有授權"。

Astra Control Center授權會使用Kubernetes CPU單元來測量CPU資源。授權必須考量指派給所有受管理Kubernetes叢集之工作節點的CPU資源。新增授權之前、您必須先從取得授權檔案（NLF） "NetApp 支援網站"。

您也可以試用Astra Control Center搭配評估授權、從下載授權之日起90天內使用Astra Control Center。您可以註冊以免費試用 "請按這裡"。

> 如果您的安裝量成長到超過授權的CPU單元數量、Astra Control Center會防止您管理新的應用程式。超過容量時會顯示警示。

當您從下載Astra Control Center時 "NetApp 支援網站"下載NetApp授權檔案（NLF）。請確定您有權存取此授權檔案。

步驟

1. 登入Astra Control Center UI。
2. 選擇*帳戶*>*授權*。
3. 選擇*新增授權*。
4. 瀏覽至您下載的授權檔案（NLF）。
5. 選擇*新增授權*。

「帳戶>*授權*」頁面會顯示授權資訊、到期日、授權序號、帳戶ID及使用的CPU單位。

> 如果您擁有評估授權、請務必儲存您的帳戶ID、以免在Astra Control Center故障時發生資料遺失（如果您未傳送ASUP）。

新增叢集

若要開始管理應用程式、請新增Kubernetes叢集、並將其當作運算資源來管理。您必須為Astra Control Center新增叢集、才能探索Kubernetes應用程式。對於Astra Data Store、您想要新增Kubernetes應用程式叢集、其中包含使用Astra Data Store所配置磁碟區的應用程式。

> 我們建議Astra Control Center先管理部署於上的叢集、再將其他叢集新增至Astra Control Center進行管理。需要管理初始叢集、才能傳送Kubmetrics資料和叢集相關資料、以供進行度量和疑難排解。您可以使用*新增叢集*功能、以Astra控制中心來管理叢集。

> 當Astra Control管理叢集時、它會追蹤叢集的預設儲存類別。如果您使用變更儲存類別 `kubectl` Astra Control會還原變更。若要變更由Astra Control管理之叢集中的預設儲存類別、請使用下列其中一種方法：
>
> - 使用Astra Control API `PUT /managedClusters` 端點、然後使用指派不同的預設儲存類別 `DefaultStorageClass` 參數。
> - 使用Astra Control Web UI指派不同的預設儲存類別。請參閱 the default storage class。

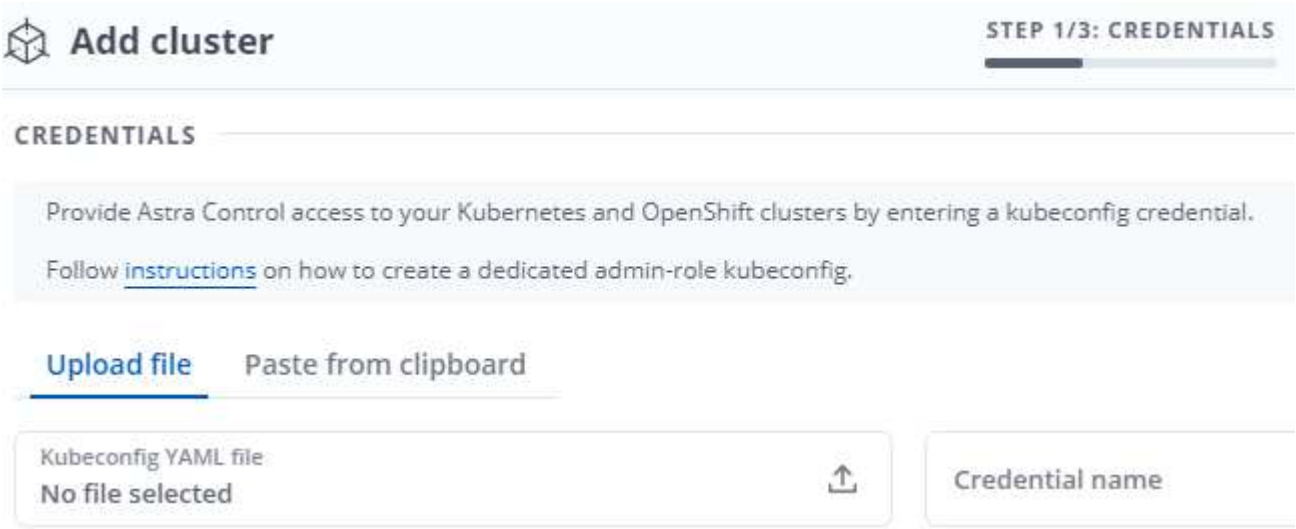
您需要的是 **#8217** ；需要的是什麼

- 新增叢集之前、請先檢閱並執行必要的 "必要工作"。

步驟

1. 在Astra Control Center UI的* Dashboard 中、選取「叢集」區段中的「Add*」。
2. 在打開的* Add Cluster-*（添加叢集）窗口中、上傳 `kubeconfig.yaml` 檔案或貼上的內容 `kubeconfig.yaml` 檔案：

   > ◦ `kubeconfig.yaml` 檔案應*僅包含一個叢集*的叢集認證資料。

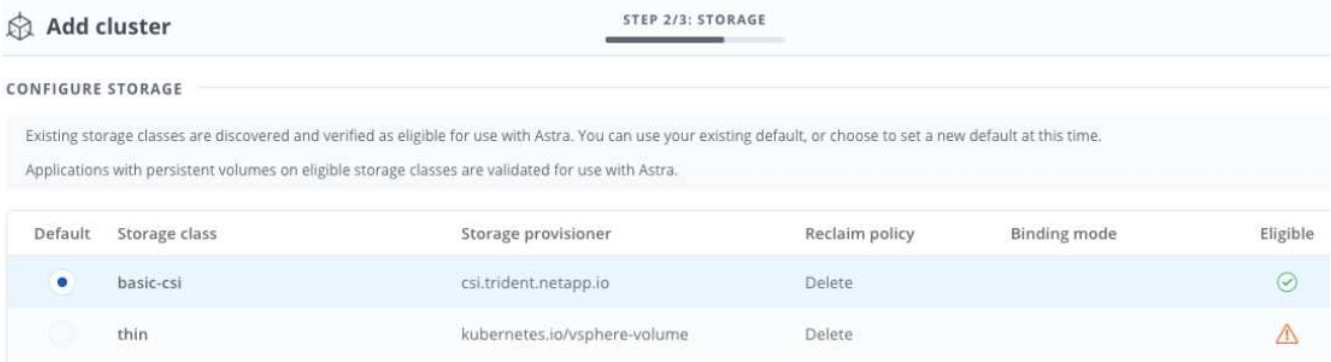


   > 如果您自行建立 `kubeconfig` 檔案中、您應該只定義*一個*內容元素。請參閱 "Kubernetes文件" 以取得有關建立的資訊 `kubeconfig` 檔案：

3. 提供認證名稱。根據預設、認證名稱會自動填入為叢集名稱。

4. 選擇*設定儲存設備*。
5. 選取要用於此Kubernetes叢集的儲存類別、然後選取* Review *。

> 您應該選擇以ONTAP 不受資料儲存或Astra Data Store支援的Trident儲存類別。



6. 檢閱資訊、如果一切正常、請選取*新增叢集*。

叢集會進入*探索*狀態、然後變更為*執行中*。您已成功新增Kubernetes叢集、現在正在Astra Control Center中進行管理。

> 在Astra Control Center中新增要管理的叢集之後、可能需要幾分鐘的時間來部署監控操作員。在此之前、通知圖示會變成紅色、並記錄*監控代理程式狀態檢查失敗*事件。您可以忽略這一點、因為當Astra Control Center取得正確狀態時、問題就能解決。如果幾分鐘內仍無法解決問題、請前往叢集並執行 `oc get pods -n netapp-monitoring` 做為起點。您需要查看監控操作員記錄、以偵錯問題。

新增儲存後端

您可以新增儲存後端、以便Astra Control管理其資源。您可以在託管叢集上部署儲存後端、或使用現有的儲存後端。

將Astra Control中的儲存叢集管理為儲存後端、可讓您在持續磁碟區（PV）與儲存後端之間建立連結、以及取得額外的儲存指標。

您的部署需求：**#8217**；現有**Astra Data Store**部署的需求

- 您已新增Kubernetes應用程式叢集和基礎運算叢集。

> 新增適用於Astra Data Store的Kubernetes應用程式叢集、並由Astra Control管理之後、叢集會顯示為 `unmanaged` 在探索到的後端清單中。接下來、您必須新增包含Astra Data Store的運算叢集、並作為Kubernetes應用程式叢集的基礎。您可以從UI的*後端*執行此動作。選取叢集的「動作」功能表、然後選取 `Manage`和 "新增叢集"。叢集狀態之後 `unmanaged` 變更Kubernetes叢集的名稱、您可以繼續新增後端。

您的部署需求：**#8217**；需要新的**Astra Data Store**部署

- 您有 "已上傳您要部署的安裝套件版本" 至Astra Control可存取的位置。
- 您已新增要用於部署的Kubernetes叢集。

- 您已上傳 a license for Astra Control Center,Astra Data Store授權 部署至Astra Control可存取的位置。

選項

- storage resources
- an existing storage backend

部署儲存資源

您可以部署新的Astra Data Store、並管理相關的儲存後端。

步驟

1. 從儀表板或後端功能表瀏覽：
    - 從*儀表板*：從「資源摘要」中、從「儲存後端」窗格中選取連結、然後從「後端」區段中選取「新增」。
    - 從*後端*：
        i. 在左側導覽區域中、選取*後端*。
        ii. 選取*「Add*」。
2. 在「部署」索引標籤中選取「* Astra Data Store*部署」選項。
3. 選取要部署的Astra Data Store套件：
    a. 輸入Astra Data Store應用程式的名稱。
    b. 選擇您要部署的Astra Data Store版本。

    如果您尚未上傳想要部署的版本、可以使用*新增套件*選項、或結束精靈並使用 "套件管理" 上傳安裝套裝組合。

4. 選取您先前已上傳的Astra Data Store授權、或使用*新增授權*選項上傳授權以搭配應用程式使用。

    具有完整權限的Astra Data Store授權會與Kubernetes叢集相關聯、而且這些相關的叢集應該會自動顯示。如果沒有託管叢集、您可以選取*新增叢集*選項、將其中一個新增至Astra Control管理。對於Astra Data Store授權、如果授權與叢集之間沒有關聯、您可以在精靈的下一頁定義此關聯。

5. 如果您尚未將Kubernetes叢集新增至Astra Control管理、則必須從* Kubernetes叢集*頁面執行此動作。從清單中選取現有的叢集、或選取*新增基礎叢集*、將叢集新增至Astra Control管理。
6. 選取Kubernetes叢集的範本大小、以提供Astra Data Store的資源。您可以選擇下列其中一項：
    - 如果您選擇 `Recommended Kubernetes worker node requirements`下、根據授權允許的範本、從大到小選取範本。
    - 如果您選擇 `Custom Kubernetes worker node requirements`下、選取每個叢集節點所需的核心數和總記憶體。您也可以顯示叢集中符合核心和記憶體選擇條件的合格節點數。

    挑選範本時、請針對較大的工作負載選擇具有較多記憶體和核心的較大節點、或針對較小的工作負載選擇較多節點。您應該根據授權允許的內容來選取範本。每個建議的範本選項都會建議符合每個節點之記憶體、核心和容量範本模式的合格節點數量。

7. 設定節點：
   a. 新增節點標籤以識別支援此Astra Data Store叢集的工作節點集區。

      在開始部署或部署之前、必須將標籤新增至叢集中用於Astra Data Store部署的各個節點。

   b. 手動設定每個節點的容量（GiB）、或選取允許的最大節點容量。
   c. 設定叢集中允許的節點數目上限、或允許叢集上的節點數目上限。
8. （僅限Astra Data Store完整授權）輸入您要用於Protection Domain的標籤金鑰。

   為每個節點的金鑰建立至少三個唯一的標籤。例如、如果您的金鑰是 `astra.datastore.protection.domain`，您可以建立下列標籤：`astra.datastore.protection.domain=domain1`,`astra.datastore.protection.domain=domain2`和 `astra.datastore.protection.domain=domain3`。

9. 設定管理網路：
   a. 輸入Astra Data Store內部管理的管理IP位址、該位址與工作節點IP位址位於同一子網路。
   b. 選擇使用相同的NIC進行管理和資料網路、或分別設定。
   c. 輸入資料網路IP位址集區、子網路遮罩和閘道、以供儲存存取。
10. 檢查組態、然後選取*部署*開始安裝。

成功安裝之後、會在中顯示後端 `available` 在後端清單中列出狀態、以及作用中的效能資訊。

您可能需要重新整理頁面、以便顯示後端。

使用現有的儲存後端

您可以將探索到ONTAP 的功能區或Astra Data Store儲存後端納入Astra Control Center管理。

步驟

1. 從儀表板或後端功能表瀏覽：
   - 從*儀表板*：從「資源摘要」中、從「儲存後端」窗格中選取連結、然後從「後端」區段中選取「新增」。
   - 從*後端*：
     i. 在左側導覽區域中、選取*後端*。
     ii. 從託管叢集的探索後端選取*管理*、或選取*新增*來管理其他現有後端。
2. 選取*使用現有*索引標籤。
3. 視後端類型而定、執行下列其中一項：
   - * Astra資料儲存區*：
     i. 選擇* Astra Data Store*。
     ii. 選取受管理的運算叢集、然後選取* Next*。

iii. 確認後端詳細資料、然後選取*「Add storage backend*（新增儲存後端*）」。

- 《》*：ONTAP

  i. 選取* ONTAP 《》《下一步》。

  ii. 輸入ONTAP 「叢集管理IP位址」和「管理認證」。

  > 您在此處輸入認證的使用者必須擁有 `ontapi` 使用者登入存取方法已在ONTAP 支援的叢集上的「支援系統管理程式」中啟用ONTAP 。如果您打算使用SnapMirror複寫、請啟用存取方法 `ontapi` 和 `http` 適用於兩ONTAP 個叢集上的使用者。請參閱 "管理使用者帳戶" 以取得更多資訊。

  iii. 選擇* Review *。

  iv. 確認後端詳細資料、然後選取*「Add storage backend*（新增儲存後端*）」。

後端隨即出現在中 `available` 列出摘要資訊。

> 您可能需要重新整理頁面、以便顯示後端。

新增儲存庫

如果您想要備份應用程式和持續儲存設備、或是想要跨叢集複製應用程式、則必須新增物件存放區資源庫供應商。Astra Control會將這些備份或複製儲存在您定義的物件存放區中。

當您新增貯體時、Astra Control會將一個貯體標示為預設的貯體指標。您建立的第一個儲存區會成為預設儲存區。

如果您要將應用程式組態和持續儲存設備複製到同一個叢集、則不需要儲存庫。

請使用下列任一種貯體類型：

- NetApp ONTAP 產品S3
- NetApp StorageGRID 產品S3
- 一般S3

  > Amazon Web Services（AWS）和Google Cloud Platform（GCP）使用通用S3儲存區類型。

- Microsoft Azure

  > 雖然Astra Control Center支援Amazon S3做為通用S3儲存區供應商、但Astra Control Center可能不支援所有聲稱Amazon S3支援的物件儲存區廠商。

- Microsoft Azure

如需如何使用Astra Control API新增儲存區的指示、請參閱 "Astra Automation和API資訊"。

步驟

1. 在左側導覽區域中、選取*鏟斗*。

a. 選取*「Add*」。

b. 選取貯體類型。

> 新增儲存庫時、請選擇正確的儲存庫供應商、並提供該供應商的適當認證資料。例如、UI接受NetApp ONTAP S3作為類型並接受StorageGRID 驗證、但這將導致所有未來使用此儲存庫的應用程式備份與還原失敗。

c. 建立新的貯體名稱、或輸入現有的貯體名稱和選用說明。

> 庫位名稱和說明會顯示為備份位置、您可以在建立備份時稍後再選擇。此名稱也會在保護原則組態期間顯示。

d. 輸入S3端點的名稱或IP位址。

e. 如果您希望此儲存區成為所有備份的預設儲存區、請檢查 `Make this bucket the default bucket for this private cloud` 選項。

> 此選項不會出現在您所建立的第一個儲存區。

f. 請繼續新增 S3 access credentials,認證資訊。

新增S3存取認證

隨時新增S3存取認證。

步驟

1. 從「庫位」對話方塊中、選取「新增」或「使用現有」索引標籤。
   a. 在Astra Control中輸入認證與其他認證不同的名稱。
   b. 從剪貼簿貼上內容、輸入存取ID和秘密金鑰。

變更預設儲存類別

您可以變更叢集的預設儲存類別。

步驟

1. 在Astra Control Center Web UI中、選取* Clusters*。
2. 在「叢集」頁面上、選取您要變更的叢集。
3. 選擇* Storage*（儲存設備）選項卡。
4. 選擇*儲存類別*類別。
5. 針對您要設為預設的儲存類別、選取「動作」功能表。
6. 選擇*設為預設*。

接下來呢？

現在您已經登入Astra Control Center並新增叢集、就可以開始使用Astra Control Center的應用程式資料管理功能。

- "管理使用者"
- "開始管理應用程式"
- "保護應用程式"
- "複製應用程式"
- "管理通知"
- "連線Cloud Insights 至"
- "新增自訂TLS憑證"

如需詳細資訊、請參閱

- "使用Astra Control API"
- "已知問題"

新增叢集的必要條件

在新增叢集之前、您應確保符合先決條件。您也應該執行資格檢查、以確保叢集已準備好新增至Astra控制中心。

新增叢集之前所需的資源

確保您的叢集符合中所述的要求 "應用程式叢集需求"。

> ⓘ 如果您計畫將第二個OpenShift 4.6、4.7或4.8叢集新增為受管理的運算資源、則應確保已啟用Astra Trident Volume Snapshot功能。請參閱官方的Astra Trident "說明" 使用Astra Trident啟用及測試Volume Snapshot。

- Astra Trident儲存類設定為 "支援的儲存後端" （任何類型的叢集都需要）
- 支援ONTAP 的支援功能不只是超級使用者和使用者ID、更能使用Astra Control Center來備份和還原應用程式。在flexf2命令列中執行下列命令ONTAP ：
  ```
  export-policy rule modify -vserver <storage virtual machine name> -policyname
  <policy name> -ruleindex 1 -superuser sysm --anon 65534
  ```
- 阿斯特拉部落 `volumesnapshotclass` 已由系統管理員定義的物件。瞭解Astra Trident "說明" 使用Astra Trident啟用及測試Volume Snapshot。
- 請確定您只為Kubernetes叢集定義單一預設儲存類別。

執行資格檢查

執行下列資格檢查、確保您的叢集已準備好新增至Astra控制中心。

步驟

1. 檢查Trident版本。

```
kubectl get tridentversions -n trident
```

如果存在Trident、您會看到類似下列的輸出：

```
NAME      VERSION
trident   21.04.0
```

如果Trident不存在、您會看到類似下列的輸出：

```
error: the server doesn't have a resource type "tridentversions"
```

> 如果未安裝Trident或安裝的版本不是最新版本、您必須先安裝最新版本的Trident、才能繼續進行。請參閱 "Trident文件" 以取得相關指示。

2. 檢查儲存類別是否使用支援的Trident驅動程式。置備程式名稱應為 `csi.trident.netapp.io`。請參閱下列範例：

```
kubectl get sc
NAME                    PROVISIONER                    RECLAIMPOLICY
VOLUMEBINDINGMODE   ALLOWVOLUMEEXPANSION   AGE
ontap-gold (default)    csi.trident.netapp.io          Delete
Immediate           true                   5d23h
thin                    kubernetes.io/vsphere-volume   Delete
Immediate           false                  6d
```

建立管理角色KECBEConfig

在執行步驟之前、請先確定機器上有下列項目：

- `kubectl` 已安裝v1.19或更新版本
- 具有作用中內容叢集管理權限的作用中Kbeconfig

步驟

1. 建立服務帳戶、如下所示：

   a. 建立名為的服務帳戶檔案 `astracontrol-service-account.yaml`。

      視需要調整名稱和命名空間。如果在此處進行變更、您應該在下列步驟中套用相同的變更。

```
<strong>astracontrol-service-account.yaml</strong>
```

+

```yaml
apiVersion: v1
kind: ServiceAccount
metadata:
  name: astracontrol-service-account
  namespace: default
```

a. 套用服務帳戶：

```
kubectl apply -f astracontrol-service-account.yaml
```

2. （選用）如果叢集使用限制的Pod安全性原則、而該原則不允許建立具有權限的Pod、或允許Pod容器內的處理程序以root使用者身分執行、請為叢集建立自訂的Pod安全性原則、讓Astra Control能夠建立及管理Pod。如需相關指示、請參閱 "建立自訂Pod安全性原則"。
3. 授予叢集管理權限、如下所示：

a. 建立 `ClusterRoleBinding` 檔案已呼叫 `astracontrol-clusterrolebinding.yaml`。

視需要在建立服務帳戶時調整任何已修改的名稱和命名空間。

```
<strong>astracontrol-clusterrolebinding.yaml</strong>
```

+

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: astracontrol-admin
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: cluster-admin
subjects:
- kind: ServiceAccount
  name: astracontrol-service-account
  namespace: default
```

a. 套用叢集角色繫結：

```
kubectl apply -f astracontrol-clusterrolebinding.yaml
```

4. 列出取代的服務帳戶機密 `<context>` 正確的安裝環境：

```
kubectl get serviceaccount astracontrol-service-account --context
<context> --namespace default -o json
```

輸出的結尾應類似於下列內容：

```
"secrets": [
{ "name": "astracontrol-service-account-dockercfg-vhz87"},
{ "name": "astracontrol-service-account-token-r59kr"}
]
```

中每個元素的索引 `secrets` 陣列開頭為0。在上述範例中、索引為 `astracontrol-service-account-dockercfg-vhz87` 將為0、索引則為 `astracontrol-service-account-token-r59kr` 應該是1。在輸出中、記下含有「權杖」一詞的服務帳戶名稱索引。

5. 產生以下的Kbeconfig：

   a. 建立 `create-kubeconfig.sh` 檔案：更換 `TOKEN_INDEX` 在下列指令碼開頭、使用正確的值。

```
<strong>create-kubeconfig.sh</strong>
```

```
# Update these to match your environment.
# Replace TOKEN_INDEX with the correct value
# from the output in the previous step. If you
# didn't change anything else above, don't change
# anything else here.

SERVICE_ACCOUNT_NAME=astracontrol-service-account
NAMESPACE=default
NEW_CONTEXT=astracontrol
KUBECONFIG_FILE='kubeconfig-sa'

CONTEXT=$(kubectl config current-context)

SECRET_NAME=$(kubectl get serviceaccount ${SERVICE_ACCOUNT_NAME} \
  --context ${CONTEXT} \
  --namespace ${NAMESPACE} \
  -o jsonpath='{.secrets[TOKEN_INDEX].name}')
TOKEN_DATA=$(kubectl get secret ${SECRET_NAME} \
```

```
  --context ${CONTEXT} \
  --namespace ${NAMESPACE} \
  -o jsonpath='{.data.token}')

TOKEN=$(echo ${TOKEN_DATA} | base64 -d)

# Create dedicated kubeconfig
# Create a full copy
kubectl config view --raw > ${KUBECONFIG_FILE}.full.tmp

# Switch working context to correct context
kubectl --kubeconfig ${KUBECONFIG_FILE}.full.tmp config use-context
${CONTEXT}

# Minify
kubectl --kubeconfig ${KUBECONFIG_FILE}.full.tmp \
  config view --flatten --minify > ${KUBECONFIG_FILE}.tmp

# Rename context
kubectl config --kubeconfig ${KUBECONFIG_FILE}.tmp \
  rename-context ${CONTEXT} ${NEW_CONTEXT}

# Create token user
kubectl config --kubeconfig ${KUBECONFIG_FILE}.tmp \
  set-credentials ${CONTEXT}-${NAMESPACE}-token-user \
  --token ${TOKEN}

# Set context to use token user
kubectl config --kubeconfig ${KUBECONFIG_FILE}.tmp \
  set-context ${NEW_CONTEXT} --user ${CONTEXT}-${NAMESPACE}-token
-user

# Set context to correct namespace
kubectl config --kubeconfig ${KUBECONFIG_FILE}.tmp \
  set-context ${NEW_CONTEXT} --namespace ${NAMESPACE}

# Flatten/minify kubeconfig
kubectl config --kubeconfig ${KUBECONFIG_FILE}.tmp \
  view --flatten --minify > ${KUBECONFIG_FILE}

# Remove tmp
rm ${KUBECONFIG_FILE}.full.tmp
rm ${KUBECONFIG_FILE}.tmp
```

b. 請輸入命令以將其套用至Kubernetes叢集。

```
source create-kubeconfig.sh
```

6. （選用）將KUbeconfig重新命名為有意義的叢集名稱。保護您的叢集認證資料。

```
chmod 700 create-kubeconfig.sh
mv kubeconfig-sa.txt YOUR_CLUSTER_NAME_kubeconfig
```

接下來呢？

現在您已經確認已符合先決條件、您已經準備好了 "新增叢集"。

如需詳細資訊、請參閱

- "Trident文件"
- "使用Astra Control API"

新增自訂TLS憑證

您可以移除現有的自我簽署TLS憑證、並以由憑證授權單位（CA）簽署的TLS憑證取代。

您需要的是 **#8217** ；需要的是什麼

- Kubernetes叢集已安裝Astra Control Center
- 管理存取叢集上要執行的命令Shell `kubectl` 命令
- 來自CA的私密金鑰和憑證檔案

移除自我簽署的憑證

移除現有的自我簽署TLS憑證。

1. 使用SSH、以管理使用者身分登入裝載Astra Control Center的Kubernetes叢集。
2. 使用下列取代命令尋找與目前憑證相關的TLS密碼 `<ACC-deployment-namespace>` 使用Astra Control Center部署命名空間：

```
kubectl get certificate -n <ACC-deployment-namespace>
```

3. 使用下列命令刪除目前安裝的機密與憑證：

```
kubectl delete cert cert-manager-certificates -n <ACC-deployment-
namespace>
kubectl delete secret secure-testing-cert -n <ACC-deployment-namespace>
```

新增憑證

新增由CA簽署的TLS憑證。

1. 使用下列命令以CA的私密金鑰和憑證檔案建立新的TLS秘密，並以適當的資訊取代括弧<>中的引數：

```
kubectl create secret tls <secret-name> --key <private-key-filename>
--cert <certificate-filename> -n <ACC-deployment-namespace>
```

2. 使用下列命令和範例編輯叢集自訂資源定義（CRD）檔案、然後變更 `spec.selfSigned` 價值 `spec.ca.secretName` 若要參考您先前建立的TLS秘密：

```
kubectl edit clusterissuers.cert-manager.io/cert-manager-certificates -n
<ACC-deployment-namespace>
....

#spec:
#  selfSigned: {}

spec:
  ca:
    secretName: <secret-name>
```

3. 使用下列命令和輸出範例來驗證變更是否正確、以及叢集是否已準備好驗證憑證、取代 `<ACC-deployment-namespace>` 使用Astra Control Center部署命名空間：

```
kubectl describe clusterissuers.cert-manager.io/cert-manager-
certificates -n <ACC-deployment-namespace>
....

Status:
  Conditions:
    Last Transition Time:  2021-07-01T23:50:27Z
    Message:               Signing CA verified
    Reason:                KeyPairVerified
    Status:                True
    Type:                  Ready
Events:                    <none>
```

4. 建立 `certificate.yaml` 使用下列範例將方括弧<>中的預留位置值取代為適當資訊的檔案：

```yaml
apiVersion: cert-manager.io/v1
kind: Certificate
metadata:
  name: <certificate-name>
  namespace: <ACC-deployment-namespace>
spec:
  secretName: <certificate-secret-name>
  duration: 2160h # 90d
  renewBefore: 360h # 15d
  dnsNames:
  - <astra.dnsname.example.com> #Replace with the correct Astra Control
Center DNS address
  issuerRef:
    kind: ClusterIssuer
    name: cert-manager-certificates
```

5. 使用下列命令建立憑證：

```
kubectl apply -f certificate.yaml
```

6. 使用下列命令和範例輸出來驗證憑證是否已正確建立、以及是否已使用您在建立期間所指定的引數（例如名稱、持續時間、續約期限及DNS名稱）。

```
kubectl describe certificate -n <ACC-deployment-namespace>
....

Spec:
  Dns Names:
    astra.example.com
  Duration:  125h0m0s
  Issuer Ref:
    Kind:        ClusterIssuer
    Name:        cert-manager-certificates
  Renew Before:  61h0m0s
  Secret Name:   <certificate-secret-name>
Status:
  Conditions:
    Last Transition Time:  2021-07-02T00:45:41Z
    Message:               Certificate is up to date and has not expired
    Reason:                Ready
    Status:                True
    Type:                  Ready
  Not After:               2021-07-07T05:45:41Z
  Not Before:              2021-07-02T00:45:41Z
  Renewal Time:            2021-07-04T16:45:41Z
  Revision:                1
Events:                    <none>
```

7. 編輯「入口CRD TLS」選項、使用下列命令和範例指向新的憑證密碼、並以適當的資訊取代方括弧<>中的預留位置值：

```
kubectl edit ingressroutes.traefik.containo.us -n <ACC-deployment-
namespace>
....

# tls:
#    options:
#      name: default
#    secretName: secure-testing-cert
#    store:
#      name: default

 tls:
    options:
      name: default
    secretName: <certificate-secret-name>
    store:
      name: default
```

8. 使用網頁瀏覽器瀏覽至Astra Control Center的部署IP位址。
9. 確認憑證詳細資料與您安裝的憑證詳細資料相符。
10. 匯出憑證並將結果匯入網頁瀏覽器中的憑證管理程式。

建立自訂Pod安全性原則

Astra Control需要在其管理的叢集上建立及管理Kubernetes Pod。如果您的叢集使用限制的Pod安全性原則、而該原則不允許建立具有權限的Pod、或允許Pod容器內的處理程序以root使用者身分執行、則您需要建立限制較少的Pod安全性原則、以啟用Astra Control來建立及管理這些Pod。

步驟

1. 為叢集建立比預設限制更少的Pod安全性原則、並將其儲存在檔案中。例如：

```yaml
apiVersion: policy/v1beta1
kind: PodSecurityPolicy
metadata:
  name: astracontrol
  annotations:
    seccomp.security.alpha.kubernetes.io/allowedProfileNames: '*'
spec:
  privileged: true
  allowPrivilegeEscalation: true
  allowedCapabilities:
  - '*'
  volumes:
  - '*'
  hostNetwork: true
  hostPorts:
  - min: 0
    max: 65535
  hostIPC: true
  hostPID: true
  runAsUser:
    rule: 'RunAsAny'
  seLinux:
    rule: 'RunAsAny'
  supplementalGroups:
    rule: 'RunAsAny'
  fsGroup:
    rule: 'RunAsAny'
```

2. 為Pod安全性原則建立新角色。

```
kubectl-admin create role psp:astracontrol \
    --verb=use \
    --resource=podsecuritypolicy \
    --resource-name=astracontrol
```

3. 將新角色繫結至服務帳戶。

```
kubectl-admin create rolebinding default:psp:astracontrol \
    --role=psp:astracontrol \
    --serviceaccount=astracontrol-service-account:default
```

版權資訊

Copyright © 2022 NetApp, Inc. 版權所有。台灣印製。非經版權所有人事先書面同意，不得將本受版權保護文件的任何部分以任何形式或任何方法（圖形、電子或機械）重製，包括影印、錄影、錄音或儲存至電子檢索系統中。

由 NetApp 版權資料衍伸之軟體必須遵守下列授權和免責聲明：

此軟體以 NETAPP「原樣」提供，不含任何明示或暗示的擔保，包括但不限於有關適售性或特定目的適用性之擔保，特此聲明。於任何情況下，就任何已造成或基於任何理論上責任之直接性、間接性、附隨性、特殊性、懲罰性或衍生性損害（包括但不限於替代商品或服務之採購；使用、資料或利潤上的損失；或企業營運中斷），無論是在使用此軟體時以任何方式所產生的契約、嚴格責任或侵權行為（包括疏忽或其他）等方面，NetApp 概不負責，即使已被告知有前述損害存在之可能性亦然。

NetApp 保留隨時變更本文所述之任何產品的權利，恕不另行通知。NetApp 不承擔因使用本文所述之產品而產生的責任或義務，除非明確經過 NetApp 書面同意。使用或購買此產品並不會在依據任何專利權、商標權或任何其他 NetApp 智慧財產權的情況下轉讓授權。

本手冊所述之產品受到一項（含）以上的美國專利、國外專利或申請中專利所保障。

有限權利說明：政府機關的使用、複製或公開揭露須受 DFARS 252.227-7013（2014 年 2 月）和 FAR 52.227-19（2007 年 12 月）中的「技術資料權利 - 非商業項目」條款 (b)(3) 小段所述之限制。

此處所含屬於商業產品和 / 或商業服務（如 FAR 2.101 所定義）的資料均為 NetApp, Inc. 所有。根據本協議提供的所有 NetApp 技術資料和電腦軟體皆屬於商業性質，並且完全由私人出資開發。 美國政府對於該資料具有非專屬、非轉讓、非轉授權、全球性、有限且不可撤銷的使用權限，僅限於美國政府為傳輸此資料所訂合約所允許之範圍，並基於履行該合約之目的方可使用。除非本文另有規定，否則未經 NetApp Inc. 事前書面許可，不得逕行使用、揭露、重製、修改、履行或展示該資料。美國政府授予國防部之許可權利，僅適用於 DFARS 條款 252.227-7015(b)（2014 年 2 月）所述權利。

商標資訊

NETAPP、NETAPP 標誌及 http://www.netapp.com/TM 所列之標章均為 NetApp, Inc. 的商標。文中所涉及的所有其他公司或產品名稱，均為其各自所有者的商標，不得侵犯。